

Control System in C on Raspberry Pi

Libraries:

```
#include <stdio.h> // printf() fflush()
#include <unistd.h> // usleep() lseek() read()
#include <stdlib.h> // system()
#include <poll.h> // poll()
#include <fcntl.h> // open()
#include <time.h> // if you need a timer in case of interval measurement
#include <pthread.h> // pthread_mutex_lock() pthread_mutex_unlock
```

To enable a couple of GPIO inputs, one interrupt the other just an input (change the numbers for your cable!):

```
system("echo 9 > /sys/class/gpio/unexport;\n\necho 9 > /sys/class/gpio/export;\n\necho in > /sys/class/gpio/gpio9/direction;\n\necho rising > /sys/class/gpio/gpio9/edge;\n\necho 10 > /sys/class/gpio/unexport;\n\necho 10 > /sys/class/gpio/export;\n\necho in > /sys/class/gpio/gpio10/direction;\n\necho none > /sys/class/gpio/gpio10/edge");
```

Using an interrupt at GPIO9:

```
struct pollfd pfd0; // the polling data structure, includes the field of file descriptor, see "man poll"
pfd0.events = POLLPRI | POLLERR; // enable the chosen events
pfd0.fd = open("/sys/class/gpio/gpio9/value", O_RDONLY | O_NONBLOCK); // open file
lseek(pfd0.fd, 0, SEEK_SET); read (pfd0.fd, str, 1); // empty the read buffer
poll(&pfd0, 1, 1000); // this will wait for the next interrupt
lseek(pfd0.fd, 0, SEEK_SET); read (pfd0.fd, str, 1); // empty the read buffer
poll again...
```

Reading a GPIO pin:

```
int fd1; // file descriptor
char str[4];
fd1 = open("/sys/class/gpio/gpio10/value", O_RDONLY | O_NONBLOCK);
lseek(fd1, 0, SEEK_SET); read (fd1, str, 1); // read the value
...//read again, etc.
```

Method for frequency measurement (encoder.c)

Two threads:

1. interrupt poll() thread increments a global variable
2. timer thread reads the global variable every 20-50ms and calculates the frequency of interrupt. The encoder provides 500 interrupts per rotation.

Method for period measurement (encoder.c)

One thread.

In a loop waits for the interrupt. Starts the timer before and reads after poll(). Very low latency, but the accuracy is low, because the interrupt latency is fluctuating 20-40us, occasionally more. With method some sort of averaging of samples is required.

Method of detecting the direction (encoder.c)

If GPIO5==1 at the time of IRQ at GPIO9, then clockwise, else anticlockwise.

Method for the regulator (regulator.c)

- A loop that reads the samples from stdin, the rate of samples determines the timing for the integrator (dt).
- The error is calculated as a difference between the reference value and the reading coming from the sensor process through stdin.
- Proportional control -- multiply the error by a coefficient.
- Integral control -- keep adding the error to a global variable (the accumulator), take dt into account. Multiply the accumulated value by a coefficient and use together with the proportional control value.
- The driver board uses the code 737 as zero input. A valid range of inputs is 737+-600.
- You may want to design experiments to determine the time constant of the motor and its sensitivity to the input. The recommended range of speeds is +-10 rotations per second (rps).